

7.2 SQL Commands

Structured Query Language (SQL) is used to extract data from databases and to insert, change, or delete new data. In addition, there are countless SQL commands to create or change new databases and to perform administrative tasks.

In this section, we'll give you some examples of how ChatGPT can help you develop correct SQL commands. The syntax of SQL is defined by standards; SQL therefore functions mostly identically in all relational DBMSs.

However, almost all DBMSs are characterized by manufacturer-specific deviations from the standard. In this respect, it's important that you always specify which program you're referring to.

7.2.1 Date Calculation

It happens time and again that you have to process dates or times. But how can you determine the orders of the past six months or the logging entries in the past three hours? How do you set a reminder for a meeting that is going to take place in a week's time? The functions for time and date calculations vary depending on the DBMS.

Prompt: MySQL: How do I get the date six months before now?

In MySQL, you can use the `DATE_SUB` function to get the date six months before the current date. Here's how you can do it:

```
SELECT DATE_SUB(NOW(), INTERVAL 6 MONTH) AS  
six_months_ago;
```

This command will return the date and time exactly six months before the current date and time. If you only need the date portion without the time, you can use `CURDATE()` instead of `NOW()`:

```
SELECT DATE_SUB(CURDATE(), INTERVAL 6 MONTH) AS  
six_months_ago;
```

7.2.2 Troubleshooting

The particularities of SQL syntax can sometimes drive you mad. AI tools succeed in solving the puzzles in many cases. In simple cases, you don't even have to tell the AI tool the schema of the database. For queries with links across multiple tables, ChatGPT and others can give more precise advice if they know the data types of the table columns.

Prompt: I want to find categories with more than 10 titles in my book library. However, my query fails (syntax error). Why is this?

```
SELECT c.name AS categoryName, COUNT(*) AS cnt  
FROM books b JOIN categories c ON c.id = b.catId  
GROUP BY c.id  
WHERE cnt >= 10
```

The issue with your query is the placement of the WHERE clause. In SQL, the WHERE clause is used before the GROUP BY clause. If you want to filter the results of a GROUP BY operation, you should use the HAVING clause. Here's how you can fix your query:

```
SELECT c.name AS categoryName, COUNT(*) AS cnt  
FROM books b JOIN categories c ON c.id = b.catId  
GROUP BY c.id  
HAVING cnt >= 10
```

We executed the second example in VS Code using GitHub Copilot. The AI tool had access to the database schema file and even stated in its response that it had used this information.

Prompt: In my accounting database, I want to create a table of all customers with the total number of invoices. However, new customers who haven't received an invoice yet are missing from the result. What's wrong with my query?

```
SELECT company, COUNT(*)  
FROM customers c JOIN invoices i on i.customerId =  
c.id  
GROUP BY c.id
```

The issue with your query is that you're using an `INNER JOIN` which only returns rows where there is a match in both tables. If a customer doesn't have an invoice yet, they won't appear in the result set.

To include customers who haven't received an invoice yet, you should use a `LEFT JOIN`. This type of join returns all the rows from the left table (customers), and the matched rows from the right table (invoices). If there is no match, the result is `NULL` on the right side.

Here's how you can modify your query:

```
SELECT c.company, COUNT(i.invoiceId)  
FROM customers c  
LEFT JOIN invoices i ON i.customerId = c.id  
GROUP BY c.id, c.company;
```

This query will return all customers, and for those without invoices, it will show a count of 0.

GitHub Copilot has correctly identified the cause of the error and has basically proposed a good solution. What is annoying here is that it has introduced a new error that could have been avoided: in `COUNT(i.invoiceId)`, the AI tool assumes that the ID column of the `invoices` table is named `invoiceId`. This may seem obvious, but in this specific case, the column is simply called `id`. This information is clear from the schema file.

7.2.3 WordPress Image Search

Recently, we wanted to add alt attributes to images in a WordPress database for an additional image description. We knew that WordPress uses MySQL or MariaDB as a DBMS, but we had never looked into the structure of this database before. ChatGPT came in handy.

Prompt: WordPress/MySQL: How can I get the newest 10 uploaded images with URL, description, and alt text from the WordPress database?

To retrieve the newest 10 uploaded images, you'll need to execute a SQL query on the WordPress MySQL database. WordPress stores information about uploaded images (and all media library items) in the `wp_posts` table and the associated metadata in the `wp_postmeta` table. The URL of an image, its description, and its alt text are stored as metadata.

The following query is an example of how to retrieve this information. This query assumes you're using the default WordPress table prefix (`wp_`). If you have a different table prefix, you'll need to replace `wp_` with your custom prefix in the query:

```
SELECT p.ID,
       p.guid AS 'URL',
       p.post_content AS 'Description',
       pm.meta_value AS 'Alt Text'
FROM   wp_posts p
LEFT JOIN wp_postmeta pm ON p.ID = pm.post_id
      AND pm.meta_key = '_wp_attachment_image_alt'
WHERE  p.post_type = 'attachment'
      AND p.post_mime_type LIKE 'image/%'
ORDER BY p.post_date DESC
LIMIT 10;
```

Here's a breakdown of what this query does . . . [more details]

Keep in mind that the `guid` field is used here to retrieve the URL, which, in general, shouldn't be used for direct URLs to media files due to potential issues with changing domains or migration. A more reliable method to get the direct URL is to use WordPress functions like `wp_get_attachment_url()` within the WordPress framework, but for direct SQL queries, the `guid` is commonly used for simplicity.

The query is helpful because it's a good starting point to understand the WordPress database setup. In a second step, we now want to determine only those images for which the `alt` attribute is missing or empty.

Prompt: OK. Instead of `p.post_content`, please include `p.post_excerpt` for the image title. Also, I want only images that don't have an alt text.

ChatGPT provides a new query that is more suitable for our purposes (not printed here). The next step is about how we can add an `alt` attribute to a given image.

Prompt: Given an ID from `wp_posts`, what would an `INSERT` statement for `wp_postmeta` look like to set the alt text for an image?

To insert an alt text for an image in the `wp_postmeta` table, given an image's ID from `wp_posts`, you would use an `INSERT` statement in SQL. The image's ID is used as the `post_id` in the `wp_postmeta` table, and the alt text is stored with a `meta_key` of `_wp_attachment_image_alt`. Here's how the `INSERT` statement would look:

```
INSERT INTO wp_postmeta (post_id, meta_key,
meta_value)
VALUES (<IMAGE_ID>,
      '_wp_attachment_image_alt',
      'Your alt text here');
```

[more details]

For our purposes, the information provided was sufficient to develop a Python script that searches for images without an image description and subsequently adds an image description to them. Perhaps you're wondering where the picture description comes from. We used an API account at OpenAI, uploaded the images there, and asked for an image description. The procedure worked amazingly well. However, the code has nothing to do with the content of this book, which is why we won't explain this example further here.

7.2.4 Complex Query with Subquery or Self Join

The starting point for the following example is a database from a cloud service provider. The monthly subscription fee is determined by the `subscriptionPrice` column. To determine the expected total monthly revenue, the

basePrice of each customer's last order must be totaled. The difficult aspect from an SQL point of view is that not all data records from the orders table are taken into account, but only the last order of each customer.

To enable ChatGPT to create a suitable SELECT query, you must inform the AI tool about the structure of the database. The easiest way to do this is to specify the CREATE-TABLE commands required to create the table. (With MySQL and MariaDB, you can determine these commands using SHOW CREATE TABLE. Another option is to create a backup of the schema definition.) In this example, we've only specified the columns relevant to the example and eliminated all other columns.

Prompt: MySQL database: I have a database with two tables customers and orders:

```
CREATE TABLE customers (  
  id                INT NOT NULL AUTO_INCREMENT,  
  company           VARCHAR(100) NOT NULL,  
  state             VARCHAR(100) NOT NULL,  
                   ON UPDATE current_timestamp(),  
  PRIMARY KEY (id),  
)  
CREATE TABLE orders (  
  id                INT NOT NULL AUTO_INCREMENT,  
  customerId         INT NOT NULL,  
  orderType          VARCHAR(20) NOT NULL,  
  orderDate          DATE NOT NULL DEFAULT curdate(),  
  subscriptionPrice DOUBLE NOT NULL DEFAULT 0,  
  PRIMARY KEY (id),  
  CONSTRAINT orders_ibfk_1 FOREIGN KEY (customerId)  
    REFERENCES customers (id)  
)
```

I need to calculate a sum over subscriptionPrice for all customers with customers.state = 'ok'. The sum should only include the newest order per customer with orders.state = 'subscription'. Can you help me with the SELECT command?

Certainly! To calculate the sum of `subscriptionPrice` for all customers with `state = 'ok'`, including only the newest order per customer with `orderType = 'subscription'`, you can use a subquery to get the newest order for each customer and then join it with the `customers` table. Here's how you can write the SQL query:

```
SELECT SUM(o.subscriptionPrice) AS total
FROM customers c
JOIN (
    SELECT customerId, MAX(orderDate) AS
latestOrderDate
    FROM orders
    WHERE orderType = 'subscription'
    GROUP BY customerId
) latest_orders ON c.id = latest_orders.customerId
JOIN orders o ON latest_orders.customerId =
o.customerId
                AND latest_orders.latestOrderDate =
o.orderDate
WHERE c.state = 'ok';
```

This query works as follows:

- The subquery `latest_orders` retrieves the `customerId` and the maximum `orderDate` for each customer where the `orderType` is `'subscription'`.
- This subquery result is joined with the `customers` table to get the customers with `state = 'ok'`.
- Another join is performed with the `orders` table to get the details of the newest order for each customer.
- Finally, the `SUM` function is used to calculate the total `subscriptionPrice` for these orders.

If you have multiple orders on the same date for a customer, this query will include all of them. If you only want one order per customer, even if there are ties, you might need a more complex query to handle tie-breaking logic.

This result is great! Not only did ChatGPT understand our intention, the AI tool also provided a pretty perfect answer and immediately pointed out the

only limitation: if a customer has placed *multiple* orders in one day, they will all be taken into account. (In this example, where it's known from AUTO_INCREMENT that the id column contains ascending values, this column could be used instead of lastOrderDate to link to the last order.)

ChatGPT has used a subquery in its suggested solution. This provides relatively readable SQL code, but is sometimes slow to execute. Even if no issues are to be expected in this example, we wanted to know whether ChatGPT also provides a solution without a subquery.

Prompt: Is it possible to do this query without a subquery?

```
SELECT SUM(o1.subscriptionPrice) AS total
FROM customers c
JOIN orders o1 ON c.id = o1.customerId
LEFT JOIN orders o2
    ON o1.customerId = o2.customerId
    AND o1.orderDate < o2.orderDate
    AND o2.orderType = 'subscription'
WHERE c.state = 'ok'
    AND o1.orderType = 'subscription'
    AND o2.id IS NULL;
```

This query works as follows [more details]

This answer is also correct. Now we wanted to know which variant is preferable. However, ChatGPT doesn't want to commit itself to this. Instead, the bot provides a comprehensive discussion of the advantages and disadvantages of both query types from various points of view.

Prompt: Which query is more efficient?

The efficiency of a query can depend on various factors, including the specific database system, indexing, and data distribution. However, in general, subqueries and self-joins have different performance characteristics.

[many more details]

7.2.5 Formatting an SQL Command

In our coding practice, we often try out a new SQL command interactively before transferring the final command to a code file later. In these experiments, we don't bother to format the SQL command in a legible way: we write the entire command in lowercase letters, don't use any line breaks, and so on.

If everything then works as expected, it's important for us to format the code properly using SQL keywords in uppercase letters, meaningful line breaks, and so on. This works very well in VS Code, where you mark the SQL code and then ask for a legible formatting via Copilot chat:

Prompt: Please capitalize SQL keywords and insert line breaks.

Alternatively, you can copy the SQL code into a chat window. We've done this here to better illustrate the concept in the book:

Prompt: Please reformat the following SQL command and capitalize all keywords.

```
create table log (id integer primary key, url text,  
ip_address text, request_time datetime);
```

```
CREATE TABLE log (  
    id INTEGER PRIMARY KEY,  
    url TEXT,  
    ip_address TEXT,  
    request_time DATETIME  
);
```

The code would be even easier to read with a few spaces between the column names and the data types, so that the data types are aligned in one column.

Prompt: Great. Could you also add spaces between the column names and their types to align the types vertically?

```
CREATE TABLE log (  
    id            INTEGER PRIMARY KEY,  
    url           TEXT,  
    ip_address    TEXT,  
    request_time  DATETIME  
);
```

Of course, this is only worthwhile for longer commands, as otherwise the prompt input takes more time than manually reformatting the code.